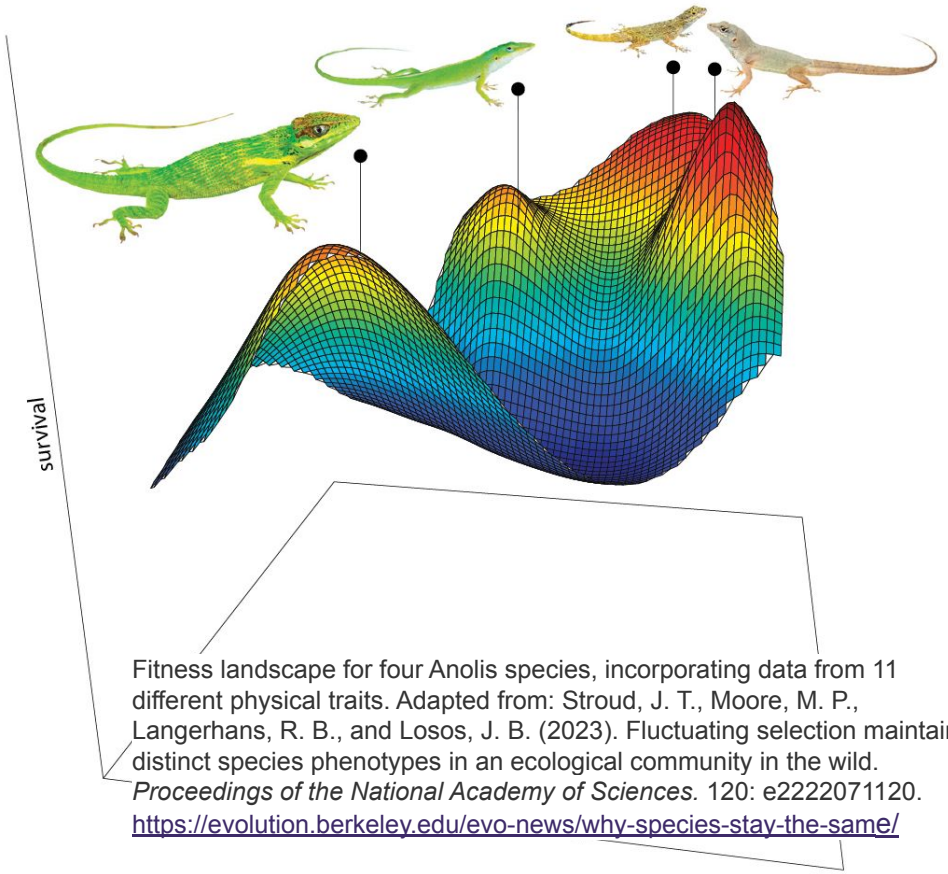
Bio-AI Assignment 3

Jon Åby Bergquist
Xavier F. C. Sánchez Díaz



Fitness landscape for four Anolis species, incorporating data from 11 different physical traits. Adapted from: Stroud, J. T., Moore, M. P., Langerhans, R. B., and Losos, J. B. (2023). Fluctuating selection maintains distinct species phenotypes in an ecological community in the wild. *Proceedings of the National Academy of Sciences.* 120: e2222071120. https://evolution.berkeley.edu/evo-news/why-species-stay-the-same/

Overview

I. Short intro to Landscape Analysis
II. Understanding the assignment
III. Step by step guide
    A. Implementing and reading tables
    B. Visualizing the landscape
    C. Visualizing Algorithm behaviour
    D. Implementation
    E. Experimentation and Comparison
    F. Testing instances

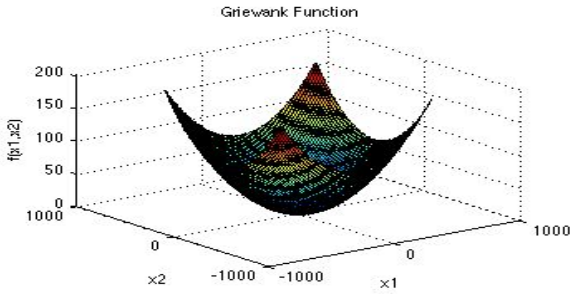
Short intro

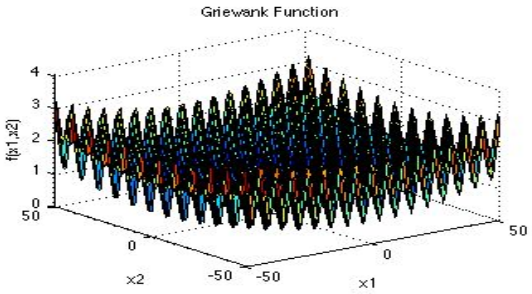
Landscape Analysis

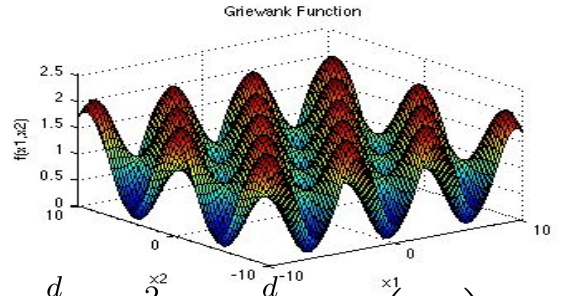
The search landscape

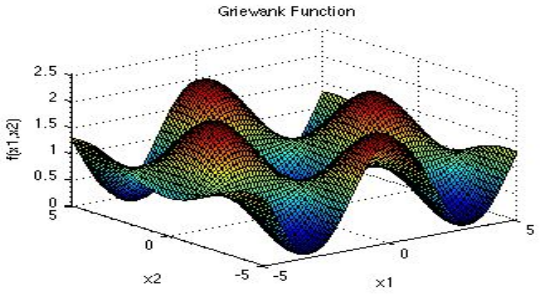
Usually, the **state space** is referred to as the **search space**.

We can **visualize** this space by looking at the objective function.





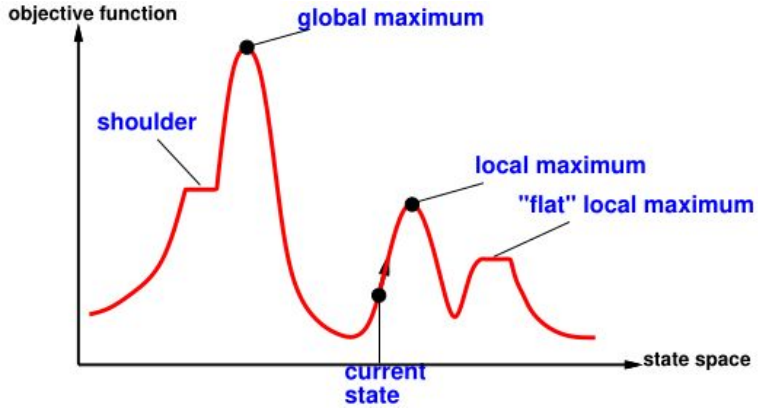




$$f(x) = \sum_{i=1}^{d} \frac{x^2}{4000} - \prod_{i=1}^{d} \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$$

The Griewank function.
Image from Surjanovic & Bingham
https://www.sfu.ca/~ssurjano/griewank.html

The search landscape

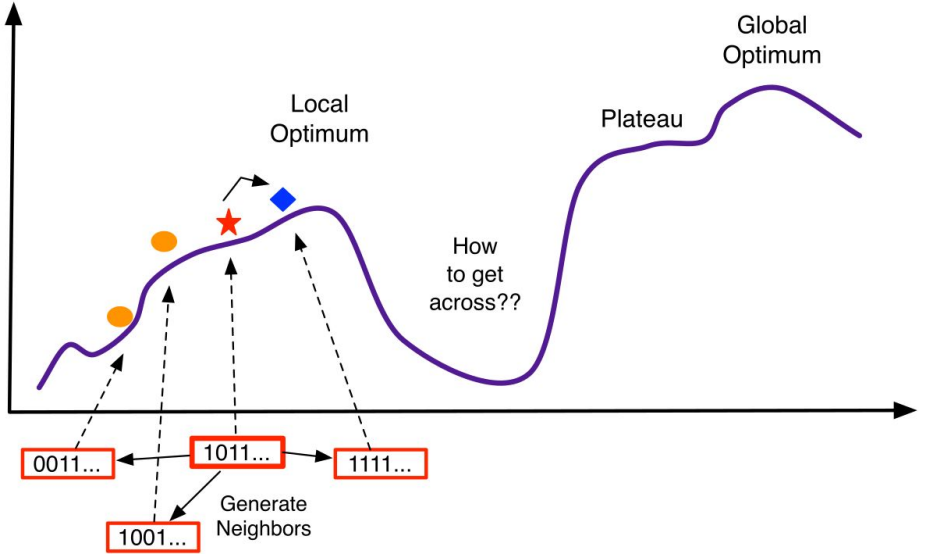


- Each point in the landscape represents a state in the search space and has *an elevation* (its f(x))
- If the elevation corresponds to an objective function, then we aim to find the highest peak (**maximum**)
- If the elevation corresponds to a cost function, then we look for the lowest point valley (**minimum**)

Exploration: Hill climbing and gradient descent

Idea: go to the best spot you see now

- Assume you are maximizing. You then want to climb the tallest peak
- This is called **hill-climbing**!



If you are **minimizing** instead, then the procedure is called **gradient descent** as we want to move towards the difference in height is largest.

Surface of selective values (Wright, 1932)

- Sewall Wright
- American geneticist
- Big name in the whole *correlation vs. causation* from the causality point of view

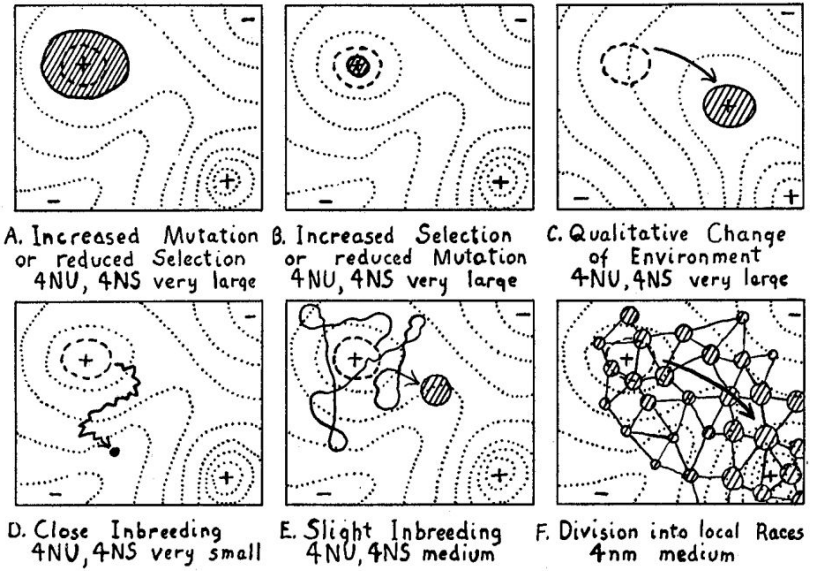
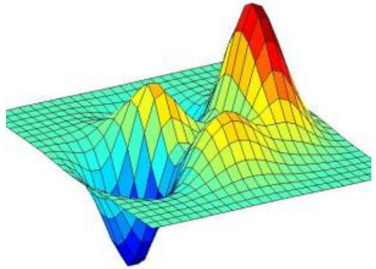


FIGURE 4.—Field of gene combinations occupied by a population within the general field of possible combinations. Type of history under specified conditions indicated by relation to initial field (heavy broken contour) and arrow.

Fitness landscapes today

We now have formalized mathematical models.



Three essential elements:

1. Search space
2. Fitness function
3. Notion of neighborhood or accessibility

Intuitively, a fitness landscape is a visualization of the terrain capturing how fitness changes between neighbouring solutions

Ideas of *valleys*, *peaks*, *ridges*, *plateaus*, *funnels*…

One fitness function has many fitness landscapes—the landscape depends on the fitness function but also on the **neighborhood** and the **sampling method**.

Beyond *fitness* landscapes

The idea of fitness landscapes has been applied in non-evolutionary contexts, so many researchers are dropping the *fitness* metaphor.

Original three elements of fitness landscapes:

- Search space
- Fitness Function
- Notion of Neighborhood or accessibility

New kinds of landscapes:

- Multi-objective landscapes
- Constraint violation landscapes
- Dynamic and coupled landscapes
- Error/loss landscapes in the context of neural networks

Search landscape analysis

Exploratory landscape analysis

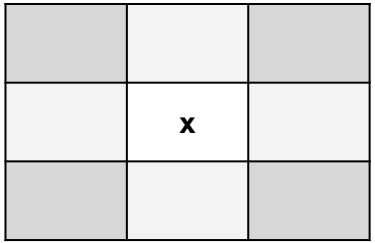
Landscape analysis

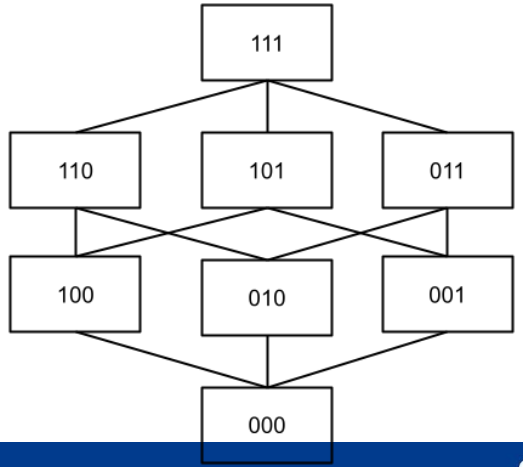
Why is this useful?

- It helps you **understand complex problems**
  - *"I cannot navigate this problem if I don't allow for unfeasible solutions to exist"*
- It allows you to **explain algorithm behavior**:
  - *"My algorithm never finds something better after this point!"*
- Can be used to **assess difficulty** rigorously:
  - For example, given the **density** and **sparsity** of **local optima**
- Used for **automated algorithm/operator selection**:
  - Landscape-aware constraint handling
  - Landscape-aware parameter setting

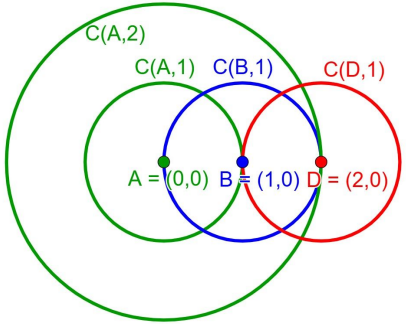
Notion of Neighborhood or accessibility

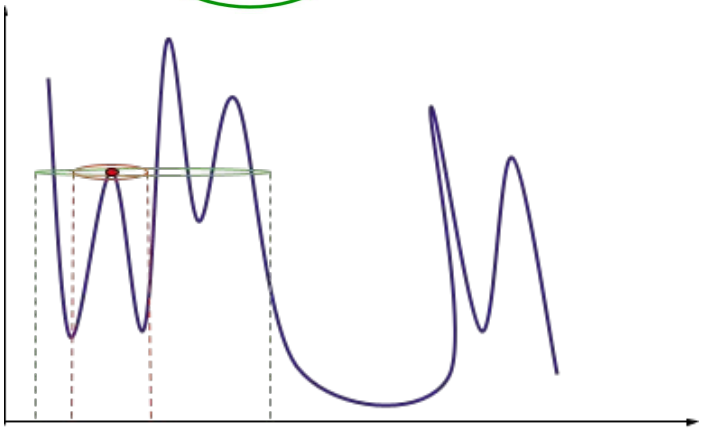
- A *necessary* definition in the landscape analysis study
- We cannot *picture* places which we cannot access
- Depends entirely on an algorithm *navigation* tactics:
  - Mutation
  - Crossover





Continuous vs. Discrete Neighborhoods



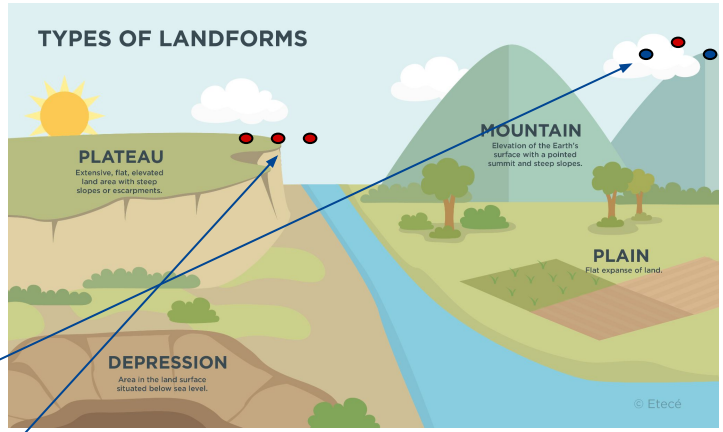


It is necessary to define a neighborhood *size* when dealing with continuous spaces.

What happens to a **local optimum** when we change the **neighborhood size**?

Local optimality

- Another crucial element in landscape analysis
- Especially important when working in continuous landscapes
- Should a solution *x* be considered a local optimum if $f(x) > x'$, $\forall\ x' \in N(x)$?
- Should it be $f(x) \geq x'$?



Plateau. Encyclopedia of humanities
https://humanidades.com/en/plateau/

Understanding the assignment

A general idea

A general idea

We're **doing science** in this project, a nice prelude to your work on the MS thesis.

- You will **compare algorithms** on different problems
- You will ask questions and draw conclusions on your findings
- You will communicate these findings to others

How?

Picture yourself playing Mario Kart:

- With the **data** and **models**, you will implement **racing tracks**
- By coding your **BioAI algorithms**, you will create different **karts**

© Nintendo https://www.gamer.no/artikler/hjula-i-taket-og-bremsespor-pa-tapeten/159874

You will **observe** a *grand prix* and **report** that!

What should I do at each step?

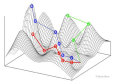
Step by step guide

Symbols used in these slides

 - Dataset

 - Vector of Labels

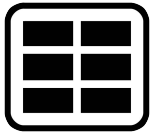
 - An ML Model

 - A Problem Landscape

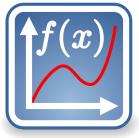
 - Lookup Table

 - Fitness Function

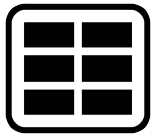
Why?



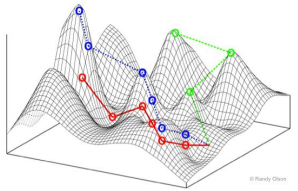
- Each row in your **table** represents one **state** (individual)
- Each **accuracy value** represents **its fitness**
  - The fitness is the “height” of the landscape



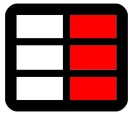
- Your **table** is a numerical representation of **a landscape**



a race track



- Your **algorithms** will **race** towards finding **its maximum**
  - Which you can easily find by looking at your table
  - The optimum is a **subset of features**
  - Its **fitness** will be the most accurate



Triangle Function

- Technical definition:
  - Looks difficult! Though it isn't that bad
- You don't have to program the function, as long as your data is the same

Definition 3.2.1. Let $\|\boldsymbol{b}\|$ be the number of 1s in the bitstring $\boldsymbol{b} \in \mathbb{B}^n$. Then, the *triangular positive wave* function, or TRIANGLE for short, is defined as:

$$\text{TRIANGLE}(\boldsymbol{b}, m, s) = \begin{cases} g(\boldsymbol{b}), & \text{if } \left\lceil \frac{\|\boldsymbol{b}\|}{s} \right\rceil \mod 2 = 1 \\ m\left(\left\lceil \frac{\|\boldsymbol{b}\|}{s} \right\rceil \cdot s - \|\boldsymbol{b}\|\right) & \text{otherwise} \end{cases} \tag{3.2}$$

where

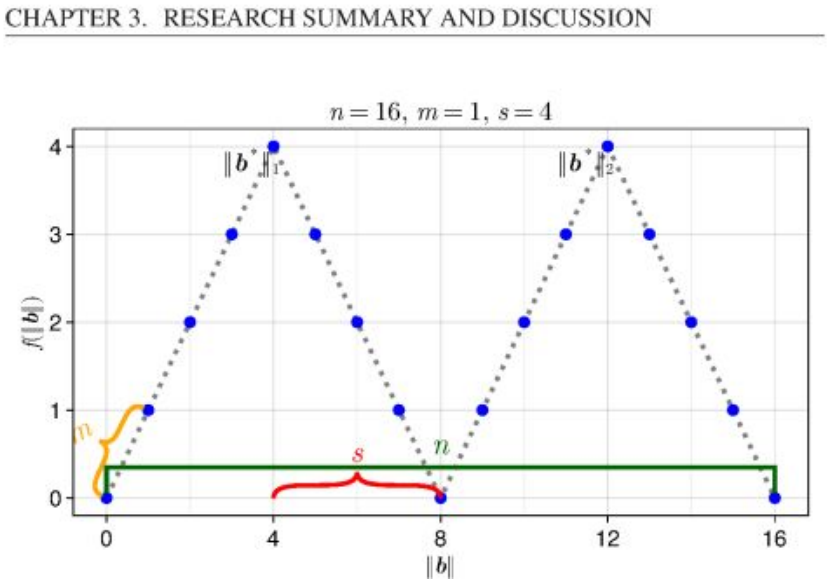
$$g(\boldsymbol{b}) = \begin{cases} m \cdot s, & \text{if } \|\boldsymbol{b}\| \mod s = 0 \\ m(\|\boldsymbol{b}\| \mod s) & \text{otherwise.} \end{cases} \tag{3.3}$$

TRIANGLE has two parameters, m, and s:

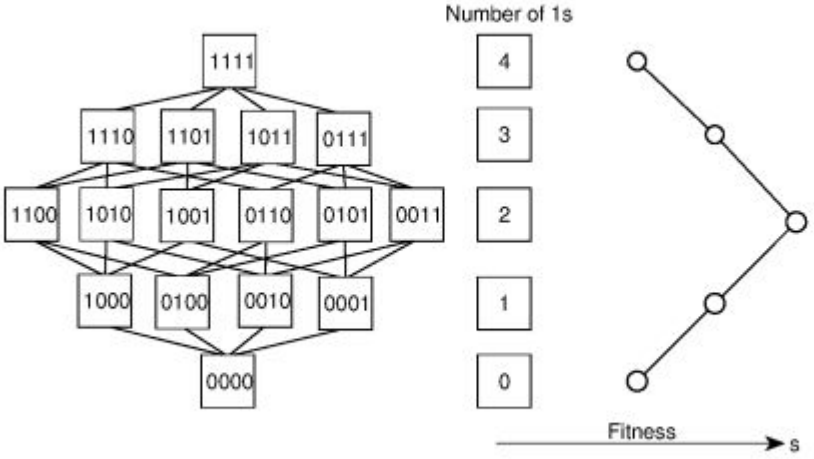
- m is a scaling factor (or *slope*) that determines the height of the peaks.
- s is a segmenting factor (or *step*) that determines the width of the peaks.

Triangle function

- For the test data
  - This is the exact shape:
- m is change in fitness for every increase in “feature”/active bits
- s is how long to step before we see changes whether we’re increasing or decreasing fitness



- Full visualization of
  - n = 4
  - m=1
  - s=2



Ask Xavier whether to show this?

```
function f(x; m=1, s=8)
     n_ones = sum(x)
     i = ceil(n_ones/s)
    if  i % 2 == 1
         if n_ones % s == 0
              r = m * s
        else
             r = m * (n_ones % s)
        end
    else
         r = m * (i * s - n_ones)
    end
    return r
end
```

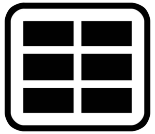
FAQs & Tips

- Do I need to use binary strings?
  - For your **lookup table**, **yes**. However, you can **use real numbers** in your **algorithm representation**, and then **round** to decide if you use a feature or not. This might be helpful for continuous algorithms!

- Can I use X programming language?
  - Yes, use whatever you like. I would not use Python as the test data will be very big. hopefully Julia (or other languages) are comfortable for you now.
  - You can use Python for visualization, I'm doing that for my research, based on results I got from C++. Advantage of Julia is that it is easy to do everything in one place.

FAQs & Tips

- What about the penalty term?
  - Since the penalty is a separate term, you can sum the penalty AFTER you have the error stored. You may want to do another column in **your table** for this.



- What are good values for the penalty?
  - 0 means no penalty
  - Too much penalty results in a flatter landscape (very hard!!)
  - Too little penalty might make the landscape very spiky (hard!!)

FAQs & Tips

- Hint for the test data:
  - Expect the big test function to be similar to the triangle function, so it is a good idea to code it so it is adaptable in the future
- If you really don’t know how to code it or how it works, you could just hardcode the fitnesses for each active “features”/n-bits in a long list, but expect that you would have to do this for 30+ values for test.
  - Although this is not a recommendation, understanding and coding, and failing is the best way to learn programming

3: Visualization

Find a nice way to **visualize** your landscapes! Suggestions:

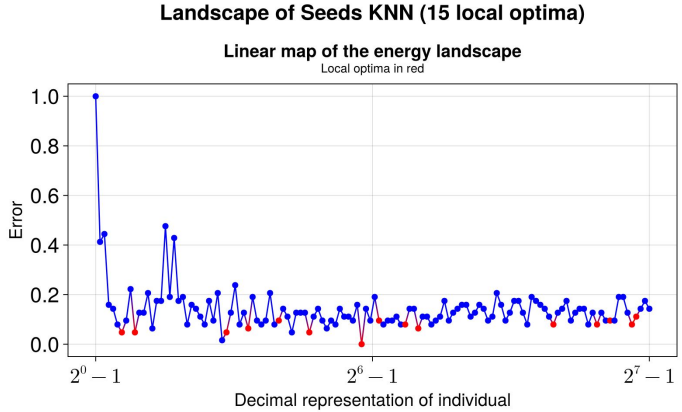
A. A 2D or 3D plot
B. A Local Optima Network (LON)
C. A Hinged Bitstring Map (HBM)
D. A Distance-Correlation map

3: Visualization

Find a nice way to **visualize** your landscapes! Suggestions:

A. **A 2D or 3D plot** **Cheap!**
   a. Difficult to find a mapping from **states** to **coordinates**
   b. But fun.

Enjoy!



3: Visualization

Find a nice way to **visualize** your landscapes! Suggestions:

Complex and Expensive!

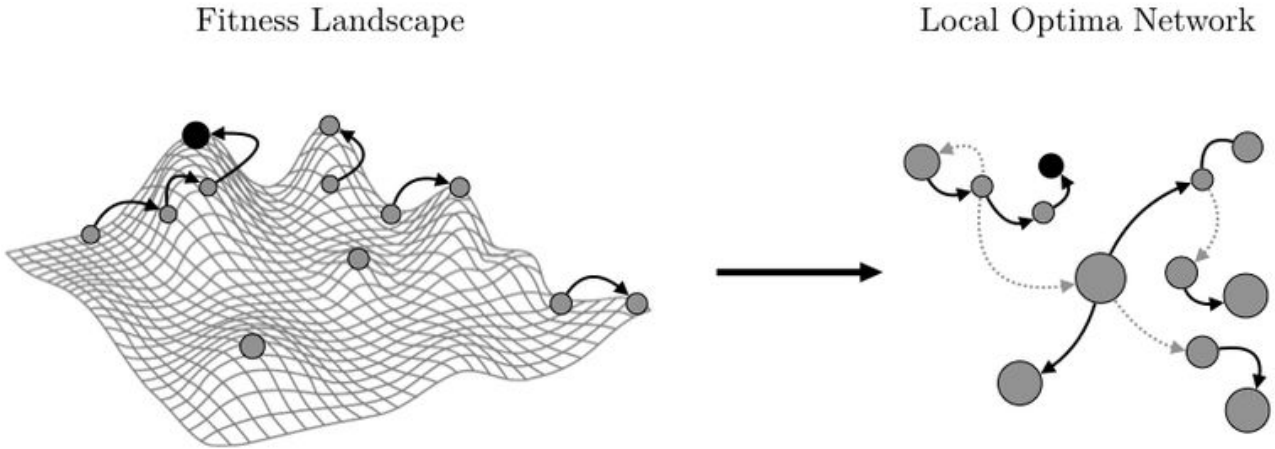
B. **A Local Optima Network (LON)**
   a. From a **local optimum**, do controlled mutation (fixed *neigh_size*)
      i. Count **how many flips** it took *to escape* that local optimum
      ii. Use this as an *estimator* of the "attraction" of this local optimum
      iii. This will be the "weight" (and size) of the node
   b. Repeat with **bigger flips**, until you can reach from one optimum to another
      i. This creates **a link** between two nodes
   c. Now repeat everything until you're done!

Read reference [6]

3: Visualization

Find a nice way to **visualize** your landscapes! Suggestions:

B. A Local Optima Network (LON)



3: Visualization

Find a nice way to **visualize** your landscapes! Suggestions:

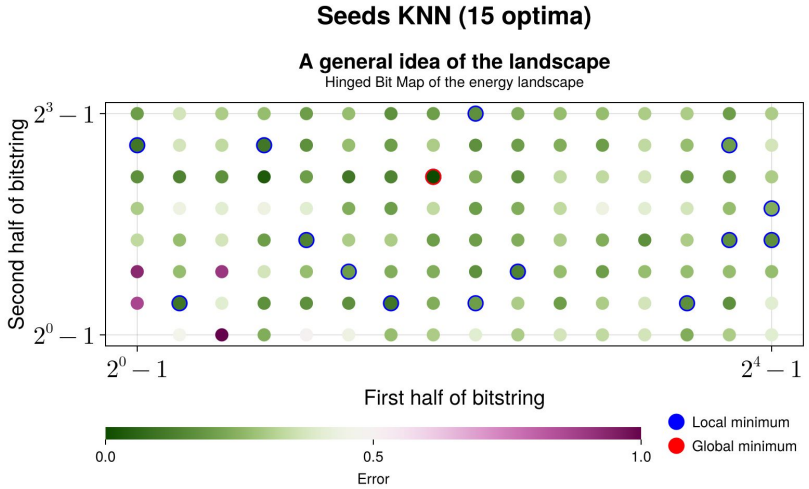
c. **A Hinged Bitstring Map (HBM)** **Complex but cheap!**
   a. Take **each bitstring** and divide in half
      i. Convert each half to base 10
      ii. Left half will be the *x*-coordinate
      iii. Right half will be the *y*-coordinate
   b. Plot as a **scatter plot**, with color=fitness
   c. Add a mark for local/global optima

Read reference [8]

3: Visualization

Find a nice way to **visualize** your landscapes! Suggestions:

c. A Hinged Bitstring Map (HBM)



NTNU

3: Visualization

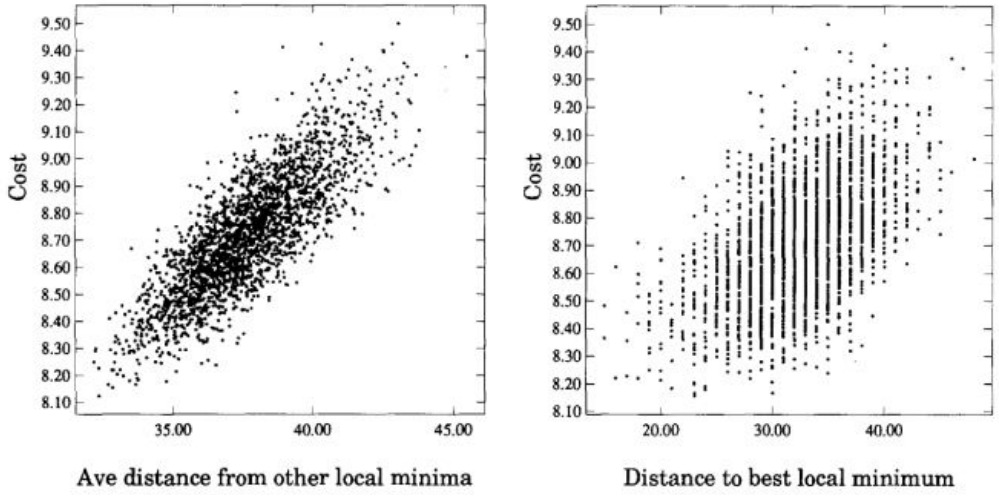
Find a nice way to **visualize** your landscapes! Suggestions:

D. **A Distance-Correlation map** **Complex!**
   a. Calculate the pairwise **distance** between **local optima (!!)**
   b. Plot the **distance** from each local optima to its nearest **global optimum** vs their **fitness**
      i. This works both as a **scatter plot** or a **hexbin plot**

3: Visualization

Find a nice way to **visualize** your landscapes! Suggestions:

D. A Distance-Correlation map



FAQs & Tips

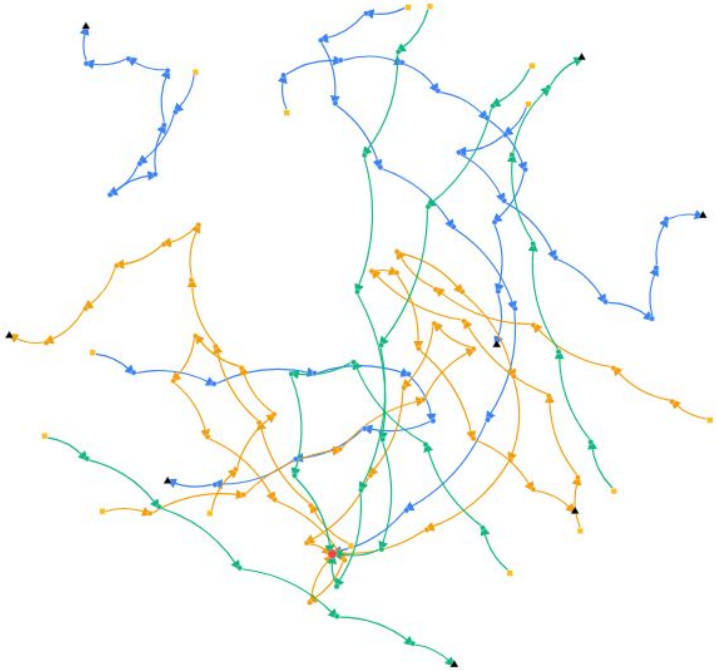
- Does it matter which visualization we use?
  - No.
- Do I have to visualize the whole landscape
  - No, for example if there are too many local optima, only show the 50 best, or ignore edges that have small thresholds. Explain why you not doing it though.
- Can I use X programming language?
  - Yes, use whatever you like. You are not expected to visualize the biggest dataset in test in full (though it may be useful). So Python is fine

4: Visualization of Algorithm behaviour

- This is Optional
- Example Methods
  - Search Trajectory Networks (STNs)
  - Attractor networks
  - You can use the methods from visualization, but changed the fitness for individual visits
    - Local Optima Networks (with highlighting)
    - HBMs with visits highlighted.
    - 2D or 3D plots
  - Entropy graphs
  - showing which features get chosen most commonly in a bar chart

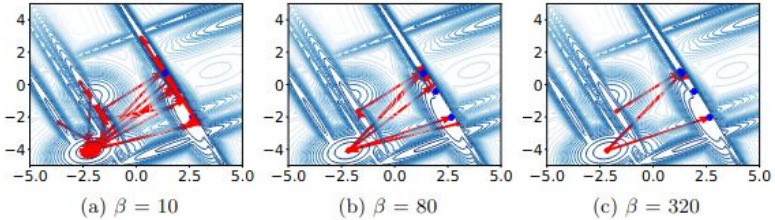
Search Trajectory Networks (optional)

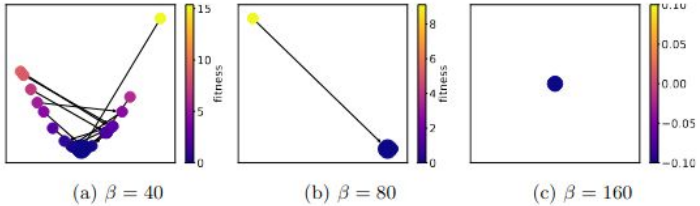
- Shows which nodes are visited, and the order
- for example highlight one individual's path
- Especially when combined with different algorithms
- experiment with: https://www.stn-analytics.com



Attractor Network (optional)

- STN's are limited, they don't show where we get stuck while searching
- Attractor network shows where the algorithm gets stuck while running.
- β is the threshold of being stuck after this many fitness evaluations





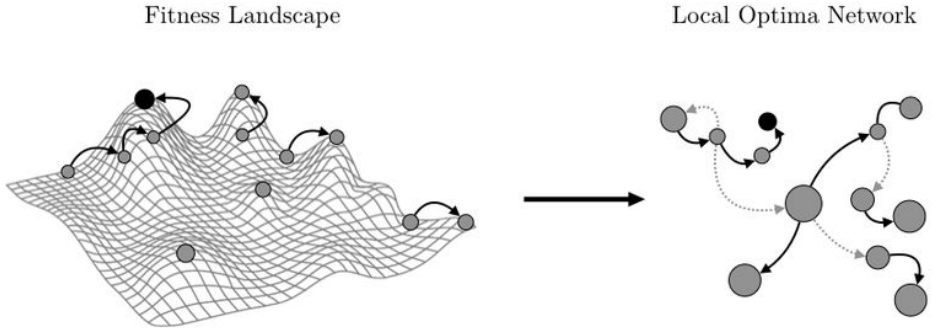
- https://arxiv.org/pdf/2412.15848

Adapting previous graphs (optional)

- Think of one specific thing you want to highlight and which landscape visualization you can adapt to highlight it
- The point is to understand what exactly is happening when running. Where is my algorithm getting stuck, which points get/don’t get visited, which features are generally chosen, what happens if I change this parameter (more mutation, more crossover, more crowding etc). Do I actually make it to the local optima, or just skirt around them

Adapting previous graphs (optional)

- Example: take a LON, instead of colouring based on fitness, do visits from algorithm.
- Use this to show whether crowding actually helps escape basins of attraction, or you just get more diversity within the same basin
- Then compare it to diversity chart
  - Am I actually escaping basins or getting diversity within a basin



Goal (optional)

- Highlight at least 3 key aspects of algorithm behaviour
- Look in project description for tips, though in general
  - Show exploration
  - show exploitation
  - compare algorithms on same landscape
  - show what changing parameters do to algorithm behaviour
  - Does the algorithm fit with landscape dynamics? AKA Expected behaviour
  - Follow the population or an individual, or a niche, many options.
- We would love if you try something new or coming up with your own techniques.

FAQs & Tips

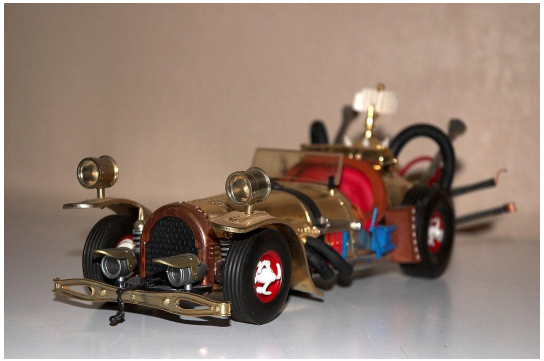
- This takes some work!
  - Yes, but it is **optional**. You don't need to do it. However it will be very useful for the test datasets
- Does it matter which visualization we use?
  - No. This task is extremely open to your ideas, in fact we encourage you to try your own ideas
- Can I use X programming language?
  - Yes, use whatever you like. You are not expected to visualize the biggest dataset in test in full (though it may be useful)

4: Implementation

Now you need to **implement your 3 BioAI algorithms**!

- One **single-objective**
- One **multi-objective**
- One from **swarm intelligence**

These are your **racing cars**



You will use these on both Step 5 and Step 6

FAQs & Tips

- Can I use my previous EAs/GAs?
  - Yes. You can use your previous algorithms (although they will need some modifications).

- Can I use the ones in EvoLP.jl?
  - Absolutely. However both the EA and the GA in EvoLP won't do any good in Step 6. Those are toy algorithms!

- How can I be sure they will be good for Step 6?
  - If they do well in your 3 landscapes, they *might* work well :)

FAQs & Tips

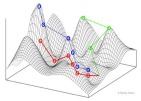
- What are the different **objectives** for the **multi-objective** case?
  - We recommend using the accuracy/error as one objective, and the penalty as a separate, second objective

- Can I use advanced versions of swarm intelligence algorithms for multiple objectives?
  - Yes, go ahead

5: Experimentation and Comparison

Experimentation is easy. It's **racing time**!

- Run your **BioAI algorithms**… 
  - *several times* each (to compensate unlucky runs!)
  - on each **landscape** using the **table values** as **fitness**



- Record your results
- **Calculate statistics** about the experiments
- Ask **questions*** and try to find the answers

FAQs & Tips

- *What kind of questions should I ask?
  - *Who won? Why?*
  - *Is one problem more difficult than the others?*
  - *What happens if you increase/reduce the penalty?*
  - *What are the hyper-parameters your algorithms need to find the solution faster? What helped the most?*

- What kind of statistics should I compute?
  - Min, average and max fitness per generation, per run, at least.
  - Average and Standard deviation of best fitness throughout runs

Check the slides from lectures in the coming weeks

The upcoming lectures will deepen into experiment handling and landscape analysis

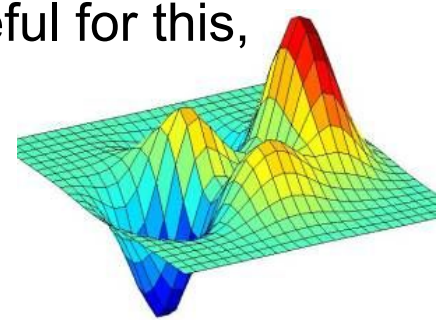
6: Test instances

You will receive **3 test "tasks"** *close* to the demo day.
There are 2 feature selection datasets similar to what you used before.
And one synthetic problem similar(ish) to the triangle function.

- Try to find the best **feature combination** represented as a bitstring, using the parameters learned on training data
- For the feature selection problem you are partially graded on how well you solve it (1pt). And explaining why you did (or didn't) solve it (1pt) (total 2 pt combined or 1 pt for each dataset)
- For the synthetic you are graded on understanding why your algorithm succeeded (or failed). (1pt)

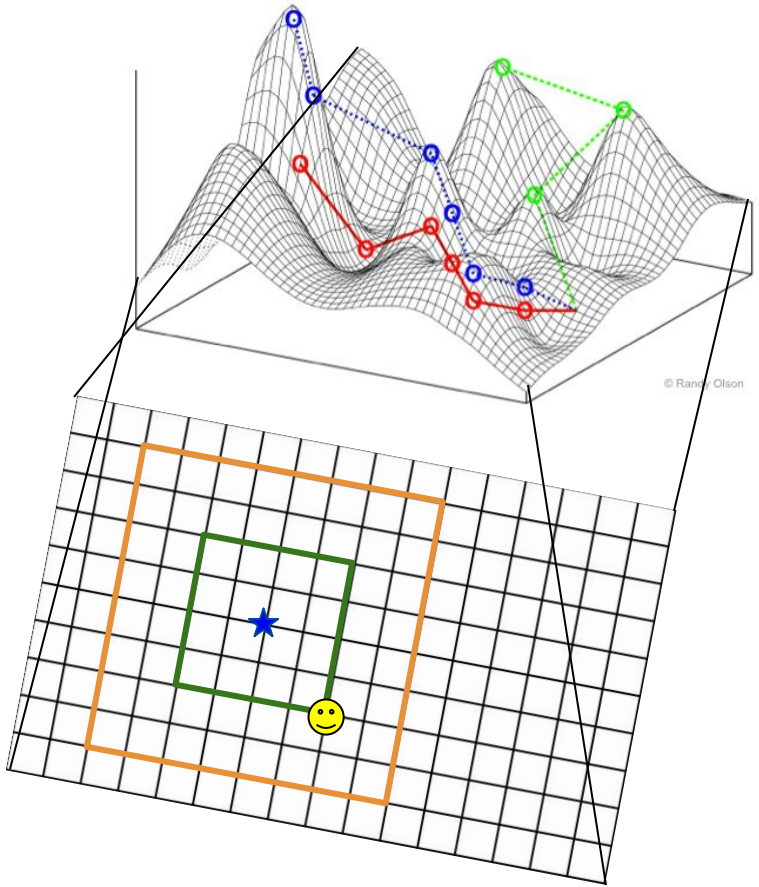
6: Test instances

- Try to find the best **feature combination** (a bitstring)
  - Using **any** of your **BioAI algorithms**
  - **Parameters frozen**, but you can use parameter control
  - Use **random initialization**
  - run, note and write down why your algorithm did or didn't solve it, and your best solution.
  - Visualization of Algorithm behaviour is very useful for this, although still optional



6: Test instances

We know the **solution**, and will calculate the **Hamming distance** between that, and your ***best* attempt**.



Illustrative picture. The landscape is not like this :)

FAQs & Tips

- The synthetic test landscape will be very big: here are some coding tips
  - Use lower precision numbers (you won’t need more than a byte as they are whole numbers below 256 in value) for this example
  - Do not store a bunch of extra values, caching visited values is useless as you have the lookup table as a full cache

Modelling and Implementation (Theoretical tips)

Reminder from our previous assignment lecture

Make it Parallel

Your laptop has **multiple cores**. Use all of them!

- **Difficulty** depends on communication complexity:
  - Multithreading: easy (and can easily halve the running time of many functions)
  - Distributed: medium (for example combining the powers of your computers and trying an island model)
  - Message Passing: hard (can create a lot of new problems)

Feedback project 2

- Better average score than project 1! 🥳
  - Still beneficial to work in teams, over 2.5 points difference!
  - Reports are much better in general
    - Use the templates provided
- As many points lost in all categories on average.
- It seems the datasets benefit from exploitation, so more focus on that would have maybe been useful if you didn't get full marks. As well as poor constraint handling. Memetic algorithms tended to do well.
- Reports
  - Mostly poor explanation and elaboration caused quite a few groups to lose some points, very few had bad structure, a lot of people were too informal.
- Questions
  - A clear pattern was that people struggled with questions related to constraint handling